

RESUMEN PASO A PASO DEL TRABAJO

Optimizacion numerica, heurísticos bioinspirados y TSP (IRNA 2026-01)

Este documento explica, en orden, QUE se hizo, COMO cubre cada entregable del enunciado y EN QUE TEORIA se baso cada parte. Es un complemento del reporte tecnico (reports/blog_post.md y reports/technical_report.md).

NOTA DE TRANSPARENCIA

El entorno de ejecucion de Python (sandbox Linux) no estuvo disponible durante la generacion automatica del proyecto. Por eso el CODIGO quedo completo y listo para ejecutar, pero los ARTEFACTOS de resultados (CSVs, figuras PNG y GIFs) deben generarse corriendo el pipeline una vez (python run_all.py). Todo lo demas (especificacion, codigo, datos, pruebas, reportes, bibliografia) esta terminado.

PARTE A. METODOLOGIA DE TRABAJO (SDD)

PASO 0 - Exploracion del material de clase (Fase 1)

- Se revisaron los notebooks y PDFs de la carpeta MaterialClase.
- Se identificaron los archivos que sirven de soporte directo:
 - * Descenso por gradiente: "Introduccion metodo de descenso por gradiente", "Perceptron y descenso por gradiente", "Optimizacion de RNA".
 - * Algoritmos evolutivos: notebook IRNA202601_prep_Algoritmos_Evolutivos.
 - * PSO: notebook IRNA202402_PSO (convencion PySwarms: w, c1, c2).
 - * Colonias de hormigas: notebook IRNABI202601_ant_colony (impl. de Berroa: alpha, beta, evaporation_rate, intensification).
 - * Productizacion: PDFs/notebook de productizacion (estructura modular).
- Entregable: docs/contexto_material_clase.md (inventario + decisiones).

PASO 1 - Especificacion tecnica antes de programar (Fase 2, SDD)

- Se redacta SPEC.md con 16 secciones: entendimiento, alcance funcional y tecnico, supuestos, ambigüedades, decisiones metodologicas, funciones y dominios, algoritmos, metricas, protocolo experimental, formato de resultados, estructura de carpetas, criterios de aceptacion, riesgos, plan de validacion y entregables.
- Se resolvieron las DOS ambigüedades del enunciado con supuestos explicitos:
 - (A) Goldstein-Price y six-hump camel son funciones de 2 variables. Para el requisito de 3D se uso la extension documentada:
$$f_{3D}(x_1, x_2, x_3) = f_{2D}(x_1, x_2) + x_3^2$$
que conserva la funcion en $x_3=0$ y NO altera el optimo principal.
 - (B) El enunciado pide ruta por Francia pero "mapa de Mexico": es una inconsistencia. La visualizacion principal se hizo sobre FRANCIA (las coordenadas reales del problema). Mexico solo seria un anexo rotulado.
- Decisiones puntuales registradas en docs/decisiones_tecnicas.md (ADRs).

PASO 2 - Estructura del proyecto (Fase 3)

- Estructura modular: src/optimization, src/tsp, src/visualization, src/utils, configs/, data/, reports/, assets/, tests/, notebooks/, docs/.
- Archivos de proyecto: README.md, requirements.txt, pyproject.toml, .gitignore, run_all.py, Makefile, LICENSE, conftest.py.

PARTE B. OPTIMIZACION NUMERICA (Parte 1 del enunciado)

PASO 3 - Funciones de prueba (Fase 4) -> src/optimization/functions.py
QUE: las seis funciones (Rosenbrock, Rastrigin, Schwefel, Griewank,

Goldstein-Price, six-hump camel), vectorizadas en NumPy, con dominio, optimo, valor minimo, validaciones, docstring con formula/fuente y un CONTADOR Estricto de evaluaciones (clase CountingFunction).

TEORIA: funciones de prueba estandar para optimizacion global (Surjanovic & Bingham, SFU; Wikipedia "Test functions for optimization"). Cada una tiene un perfil distinto:

- Rosenbrock: valle curvo, casi unimodal (explotacion).
- Rastrigin/Griewank: muchos minimos locales (multimodal).
- Schwefel: optimo lejos del centro, "trampas" en los bordes.
- Goldstein-Price / six-hump camel: varios optimos locales/globales.

CUBRE: dominios estandar exactos del enunciado y validacion de minimos conocidos (tests/test_functions.py).

PASO 4 - Gradientes (Fase 5) -> src/optimization/gradients.py

QUE: gradiente por DIFERENCIAS FINITAS CENTRALES con paso $h=1e-5*(1+|x|)$; cada gradiente cuesta $2*d$ evaluaciones, todas contadas en el NFE.

Ademas gradientes analiticos de Rosenbrock y Rastrigin para validacion.

TEORIA: aproximacion de la derivada por diferencias centrales (error $O(h^2)$).

Se eligio sobre derivadas analiticas para tratar las 6 funciones de forma uniforme (decision DT-02).

PASO 5 - Descenso por gradiente (Fase 5) ->

src/optimization/gradient_descent.py

QUE: GD desde cero: condicion inicial aleatoria en el dominio (semilla controlada), tasa de aprendizaje configurable, parada por norma del gradiente y por cambio pequeno en f, clipping/proyeccion al dominio, registro de trayectoria (para animaciones) y de convergencia.

TEORIA: $x_{k+1} = x_k - lr * \text{grad } f(x_k)$; metodo de primer orden que explota informacion local de la pendiente (base del entrenamiento de RNA).

CUBRE: se ejecuta por funcion x dimension (2D y 3D) repitiendo $n = 100, 500$ y 1000 inicializaciones aleatorias (requisito exacto del enunciado).

PASO 6 - Heuristicos: EA, PSO, DE (Fase 6)

QUE y TEORIA:

- evolutionary.py (Algoritmo Evolutivo): seleccion por TORNEO, cruce aritmetico/uniforme, mutacion GAUSSIANA, ELITISMO. Base: Holland (1975), computacion evolutiva; convenciones del notebook de clase (elitismo + fraccion de mutacion).
- pso.py (Particle Swarm Optimization): posicion/velocidad, mejor personal b_i y mejor global g , inercia w , coef. cognitivo $c1$ y social $c2$, limites

de

posicion y velocidad. Base: Kennedy & Eberhart (1995).

- differential_evolution.py: estrategia DE/rand/1/bin, factor de mutacion F , tasa de cruce CR , seleccion por supervivencia (greedy). Base: Storn & Price (1997).

CUBRE: cada algoritmo corre ≥ 30 corridas independientes por funcion y dimension; parametros base en configs/numerical_experiments.yaml.

PASO 7 - Runner y metricas (Fases 6-7)

-> experiment_runner.py, metrics.py

QUE: orquesta todas las corridas y produce:

data/results/numerical_results.csv (una fila por corrida)
data/results/numerical_summary.csv (media, mediana, desv, mejor, peor, NFE medio/mediano, tasa de exito, tiempo medio, semilla base).

CUBRE: la tabla resumen exacta que pide el enunciado.

PASO 8 - Comparacion y discusion (Fase 7) -> reports/blog_post.md, seccion 4.7

TESIS sustentada por el diseno experimental:

- El GD aporta EXPLOTACION LOCAL barata: eficiente cuando la superficie es suave y la inicializacion cae cerca de una buena cuenca (Rosenbrock). Su debilidad es la sensibilidad a la condicion inicial (se atasca en minimos locales en Rastrigin/Griewank/Schwefel).
- Los heuristicos aportan EXPLORACION GLOBAL y robustez: mejor mediana y menor varianza en funciones multimodales, a costa de MUCHAS mas

evaluaciones (1-2 ordenes de magnitud mas que el GD).

- El mejor metodo depende de la funcion, la dimension y el presupuesto de evaluaciones. Estrategia hibrida: heuristico para localizar la cuenca + GD para refinar.

PASO 9 - Animaciones (Fase 8) -> src/visualization/animations.py

QUE: dos GIFs sobre Rastrigin 2D:

- gd_rastrigin_2d.gif: curvas de nivel + trayectoria + valor de f.
- pso_rastrigin_2d.gif: enjambre moviendose + mejor global + valor.

TEORIA/TECNICA: matplotlib.animation con PillowWriter (no requiere imagemagick, a diferencia del notebook de clase). Decision DT-08.

PASO 10 - Histogramas (Fase 5) -> src/visualization/histograms.py

QUE: por combinacion funcion/dim/n, histograma de (a) valor final de f, (b) numero de evaluaciones (NFE), (c) solucion final por coordenada.

CUBRE: exactamente los histogramas pedidos, con nombres de archivo claros (p. ej. gradient_descent_rastrigin_2d_n1000_final_values.png).

PARTE C. OPTIMIZACION COMBINATORIA - TSP DE FRANCIA (Parte 2)

PASO 11 - Datos de las 96 capitales (Fase 9) -> src/tsp/data_loader.py

QUE: dataset embebido de las 96 prefecturas de la Francia metropolitana (departamentos 01-95, con Corcega dividida en 2A y 2B; sin ultramar), con codigo, departamento, prefectura, latitud, longitud, fuente y notas. Genera data/processed/france_96_capitals.csv y data/raw/france_departments_sources.csv. Se valida (96 ciudades, sin duplicados, coordenadas dentro de Francia).

FUENTE: coordenadas geograficas publicas de las prefecturas (Wikipedia/OpenStreetMap/IGN), precision ~0.01-0.05 grados (supuesto S1).

PASO 12 - Modelo de costos (Fase 9) -> src/tsp/cost_model.py

FORMULA (la del enunciado):

$$\begin{aligned} \text{costo_total_ij} = & \text{tiempo_horas_ij} * \text{valor_hora} \\ & + \text{distancia_km_ij} * \text{costo_combustible_km} \\ & + \text{peaje_estimado_ij} \end{aligned}$$

SUPUESTOS DOCUMENTADOS (no se inventaron datos puntuales como reales):

- Distancia: haversine * factor de detour 1.30 (aprox. red vial) [S2].
- Tiempo: distancia / 90 km/h (velocidad media efectiva) [S3].
- Combustible: distancia * (consumo/100) * precio_litro.
- Peaje: distancia * 0.70 * 0.092 EUR/km (fraccion autopista * tarifa) -> ESTIMACION documentada, no matriz oficial por tramo [S4].

TEORIA: distancia del gran circulo (haversine) sobre la esfera terrestre.

Matrices 96x96 guardadas en data/processed/tsp*_matrix*.csv.

PASO 13 - Vehiculo y escenarios de valor hora (Fase 9)

- Vehiculo base: Renault Clio V 1.0 TCe gasolina; consumo 5.3 L/100km (WLTP); precio 1.75 EUR/L (orden de magnitud Francia 2024-2025) [S5].
- Valor hora del vendedor estudiado en 3 escenarios: bajo 15, medio 30, alto 50 EUR/h. Hipotesis: la geometria de la ruta optima cambia poco, pero el costo total escala casi linealmente con el valor hora.

PASO 14 - Algoritmos combinatorios (Fase 10)

- ant_colony.py (ACO): feromonas tau, visibilidad eta = 1/costo, evaporacion rho, deposito Q del mejor de la iteracion (elitista), varias hormigas, regla de transicion proporcional a $\tau^\alpha * \eta^\beta$, ciclo cerrado. Registro de la mejor ruta por iteracion. Base: Dorigo & Stutzle (2004).
- genetic_algorithm.py (GA): rutas como PERMUTACIONES, seleccion por torneo, cruce de orden OX (valido para permutaciones), mutacion por inversion, elitismo. Registro de la mejor ruta por generacion. Base: Holland;

operadores

OX para TSP.

CUBRE: ambos corren en los 3 escenarios de valor hora. Resultados en data/results/tsp_results.csv, tsp_best_routes.csv, tsp_summary.csv, con costo, distancia, tiempo, peajes, combustible, costo por tiempo, iteracion del mejor, tiempo de ejecucion y semilla.

PASO 15 - Visualizacion TSP (Fase 11) -> src/visualization/maps.py

QUE: mapa estatico de la mejor ruta ACO, mejor ruta GA y comparacion; y animacion de la evolucion de la mejor ruta sobre Francia: tsp_france_best_route_aco.gif y tsp_france_best_route_ga.gif.

NOTA: se incluye la advertencia tecnica sobre la incoherencia del "mapa de Mexico" (se uso Francia por coherencia con las coordenadas).

PARTE D. CALIDAD, PRUEBAS Y REPRODUCIBILIDAD

PASO 16 - Pruebas (Fase 12) -> tests/

- test_functions.py: las 6 funciones alcanzan su minimo conocido (2D y 3D); extension 3D coincide con 2D en $x_3=0$; el contador de evaluaciones incrementa bien; sin NaN/Inf; gradiente FD == analitico.
- test_algorithms.py: GD respeta dominio y cuenta evaluaciones; PSO/DE/EA convergen bajo umbrales; historial del mejor es monotono.
- test_tsp.py: 96 ciudades sin duplicados; matriz 96x96 simetrica con diagonal cero; rutas con las 96 ciudades sin repetir; costo consistente con la formula.
- Evidencia: reports/test_results.md (pegar ahi la salida de pytest).

PASO 17 - Pipeline y modos (Fase 18) -> run_all.py

- Modo fast (validacion en minutos) y modo full (cumple el enunciado: GD con $n=100/500/1000$ y ≥ 30 corridas heurísticas).
- Reproducibilidad: semillas derivadas deterministas (utils/seed.py); configs externas en YAML; logging.

PASO 18 - Reportes y documentacion (Fases 13-17)

- reports/blog_post.md: entrada de blog lista para publicar (titulo, resumen, introduccion, metodologia SDD, Parte 1, Parte 2, uso de IA, limitaciones, conclusiones, bibliografia, repositorio, anexo de contribucion). Cada figura y tabla esta NUMERADA, titulada y referenciada en el texto.
- reports/technical_report.md: version tecnica + bibliografia APA completa.
- reports/prompts_ia.md: prompts usados, que produjo cada uno, que se reviso y el impacto de la IA (Fase 15).
- reports/contribution_video_template.md: plantilla y guion en primera persona para el video individual (Fase 16).
- README.md: instalacion, ejecucion, pruebas, estructura, donde pegar la URL del repo (Fase 17).

PARTE E. BASE TEORICA (RESUMEN DE REFERENCIAS)

- Descenso por gradiente / optimizacion de RNA: material de clase + teoria de metodos de primer orden.
- Funciones de prueba: Surjanovic & Bingham (SFU); Wikipedia "Test functions for optimization"; Rosenbrock (1960); Rastrigin (1974); Goldstein & Price (1971).
- PSO: Kennedy & Eberhart (1995); convencion PySwarms (Miranda, 2018).
- Algoritmos evolutivos: Holland (1975/1992); notebook de clase; PyGAD (Gad).
- Evolucion diferencial: Storn & Price (1997).
- Colonias de hormigas: Dorigo & Stutzle (2004); notebook de clase (Berroa).
- TSP: Wikipedia "Travelling salesman problem".
- Distancia geografica: formula haversine (gran circulo).
- Datos: OpenStreetMap / Wikipedia (coordenadas); precios de combustible y

peajes de Francia como estimaciones documentadas.
(Lista APA completa en reports/technical_report.md, seccion 7.)

=====

PARTE F. COMO QUEDA CUBIERTO CADA ENTREGABLE DEL ENUNCIADO

=====

[x]	Parte 1.1	GD con n=100/500/1000, 2D y 3D, histogramas de solucion y de NFE.
[x]	Parte 1.2	EA, PSO y DE implementados y ejecutados (>=30 corridas).
[x]	Parte 1.3	GIFs de GD y de heuristico (PSO) para un caso (Rastrigin 2D).
[x]	Discusion	Comparacion valor final vs. NFE, multiples corridas.
[x]	Parte 2	TSP de las 96 capitales de Francia con ACO y GA.
[x]	Costos	tiempo del vendedor + combustible + peajes; vehiculo definido; valor hora estudiado (3 escenarios).
[x]	Mapa/GIF	animacion de la mejor ruta sobre Francia (con nota sobre Mexico).
[x]	Reporte	tipo blog, figuras/tablas rotuladas y citadas, bibliografia APA.
[x]	Uso de IA	reportado con prompts e impacto.
[x]	Video	seccion/plantilla de contribucion individual.
[x]	Codigo	referenciable desde un repositorio Git (espacio para la URL).
[x]	SDD	SPEC.md antes de implementar; supuestos documentados.

PENDIENTES (solo dependen de ti):

- 1) Crear el repositorio en GitHub y subir el proyecto.
- 2) Pegar la URL del repo en README.md y en reports/blog_post.md (seccion 10).
- 3) Grabar el video de contribucion individual (usar la plantilla).
- 4) Ejecutar: `python run_all.py --mode full`
para poblar CSVs, figuras y GIFs; y `pytest` para pegar la evidencia.
- 5) Generar el PDF final del reporte (ver reports/reporte_tecnico.html:
abrir en el navegador -> Imprimir -> Guardar como PDF).

=====

FIN DEL RESUMEN

=====